

ELRS Ground Station

Benutzerhandbuch

Ein schlanker Ground-Control-Bildschirm für
ExpressLRS (ELRS) Modelle

Stand: August 2026

github.com/KresserSimon/ELRS_Telemetry_Groundcontrol

Inhalt

- 1. Einleitung
- 2. Installation
- 3. Erste Schritte
- 4. Verbindung zur Telemetrie herstellen (inkl. Antennen-Tracker-Ausgabe, Modell-Profil)
- 5. Offline-Nutzung (Longrange ohne Internet)
- 6. Die Benutzeroberfläche
- 7. Kartenoptionen (Vektorkarte als Standard, Satellitenbild/OpenStreetMap, Sperrzonen inkl. OpenAIP, Rechtsklick-Menü, Home-Position)
- 8. Route/Wegpunkte planen, Höhenprofil, Grid-Muster, INAV-Mission-Export
- 9. Flugpfad-Aufzeichnung (Start/Pause/Export)
- 10. Fluglog (CSV-Aufzeichnung)
- 11. Plan-Modus
- 12. Die Menüs im Detail
- 13. Akkuwarnung: LiPo vs. Li-Ion
- 14. Als eigenständige .exe kompilieren
- 15. Anhang: Kommandozeilen-Referenz

1. Einleitung

ELRS Ground Station ist eine leichtgewichtige Alternative zu Mission Planner oder QGroundControl für Modelle mit ExpressLRS (ELRS). Die App zeigt live auf einer Vektorkarte (Standard) oder einer OpenStreetMap-/Satellitenkarte, wo sich das Modell befindet, stellt alle wichtigen Telemetriedaten (GPS, Akku, Funkverbindung, Sensoren) in einem übersichtlichen Dashboard dar, warnt per Sprachausgabe bei niedrigem Akkustand und erlaubt es, Flugpfade aufzuzeichnen, Routen/Wegpunkte zu planen und als INAV-Mission zu exportieren.

Die App funktioniert mit zwei Telemetrie-Protokollen:

- **MAVLink** - ausgegeben von Flugsteuerungen wie ArduPilot, iNav oder Betaflight, die per CRSF-Telemetrie mit dem ELRS-Empfänger verbunden sind.
- **CRSF (Crossfire)** - das native Telemetrieprotokoll von ExpressLRS. CRSF wurde ursprünglich von TBS (Team BlackSheep) für deren Crossfire-Funksystem entwickelt; ExpressLRS verwendet bewusst dasselbe Frameformat, sodass die App auch mit echter TBS-Crossfire-Hardware funktioniert.

Beide Protokolle lassen sich sowohl über WiFi (UDP) als auch über eine direkte USB/seriell-Verbindung empfangen, umschaltbar zur Laufzeit ohne Neustart der App.

2. Installation

Voraussetzung ist Python 3.10 oder neuer. Im Projektordner:

```
cd elrs_ground_station
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

Die requirements.txt installiert PyQt6 (inkl. WebEngine für die Karte und WebChannel für die Interaktion mit der Karte), pymavlink, pytsx3 (Sprachausgabe), pyserial (USB-Verbindungen) und pmtiles (Lesen/Extrahieren von Vektorkarten-Regionsdateien). Unter Windows nutzt pytsx3 die eingebaute SAPI5-Sprachausgabe - es sind keine zusätzlichen Systempakete nötig.

Alternativ steht eine fertig kompilierte .exe zur Verfügung (siehe Abschnitt 14), die ohne Python-Installation läuft.

3. Erste Schritte

Zum unverbindlichen Ausprobieren ohne jegliche Hardware:

```
python main.py --demo
```

Der Demo-Modus simuliert einen Loiter-Kreisflug inklusive Akku-Entladung, Roll/Pitch-Bewegung, wechselnden Flugmodi und schwankender Funkverbindung, sodass sich alle Funktionen der App - einschließlich der Sprachwarnung bei niedrigem Akku - ohne echtes Modell testen lassen.

Wird die App ohne --demo gestartet, öffnet sich zunächst ein Popup zur Auswahl von Verbindung (WiFi/UDP oder USB) und Protokoll (MAVLink oder CRSF). Ein Klick auf Abbrechen übernimmt einfach die per Kommandozeile übergebenen bzw. die Standard-Einstellungen; zwei eigene Buttons im Popup starten stattdessen direkt den Demo- bzw. den Plan-Modus (siehe Abschnitt 11) - ohne jede Telemetrie-Verbindung.

4. Verbindung zur Telemetrie herstellen

ELRS-Hardware spricht kein natives 'Telemetrie-über-WiFi' - das eingebaute WiFi eines ELRS-Moduls dient primär dem Flashen/Konfigurieren (Access Point ExpressLRS TX/ExpressLRS RX, Standardpasswort expresslrs). Um Telemetrie tatsächlich zu dieser App zu bekommen, gibt es drei Wege:

Weg 1: MAVLink über WiFi (empfohlen)

Voraussetzung ist eine Flugsteuerung mit MAVLink-Ausgabe, die per CRSF-Telemetrie mit dem ELRS-Empfänger verbunden ist. Der serielle MAVLink-Strom der Flugsteuerung muss per WiFi-Bridge (z. B. ein ESP32/ESP8266 mit MAVESP8266-Firmware) auf UDP-Port 14550 gebracht werden.

```
python main.py --protocol mavlink --host 0.0.0.0 --port 14550
```

Muss die Bridge stattdessen aktiv eine Verbindung zum PC aufbauen: `--udp-mode connect --host <IP-der-Bridge>`

Weg 2: Rohes CRSF/TBS-Crossfire über WiFi

Manche ELRS-'Backpack'-Bridges (ESP32-basiert) leiten den rohen CRSF-Bytestrom direkt per UDP weiter, ohne Umweg über MAVLink.

```
python main.py --protocol crsf --host 0.0.0.0 --port 14551
```

Dieser Modus deckt GPS (inkl. Geschwindigkeit), Akku (Spannung/Strom/Restkapazität/verbrauchte mAh), Link-Statistiken, Attitude (Roll/Pitch) sowie - falls gesendet - Vario, Baro-Höhe, RPM, Temperatur und Zellspannungen ab.

Weg 3: USB/seriell

Flugsteuerung oder ELRS TX-Modul per USB-Kabel direkt an den PC anschließen. Verfügbare Ports zunächst auflisten:

```
python main.py --list-ports
python main.py --connection usb --protocol mavlink --serial-port COM5
python main.py --connection usb --protocol crsf --serial-port COM5 --baud 420000
```

Standard-Baudrate: 57600 für MAVLink, 420000 für CRSF (jeweils per `--baud` überschreibbar). Diese Verbindungsart ersetzt Weg 1/2, ist kein Zusatz dazu.

In allen WiFi-Fällen müssen PC und Bridge/Modul im selben Netzwerk sein. Alle drei Wege lassen sich auch nachträglich über Telemetrie & Hardware -> Verbindung... ändern, ohne die App neu zu starten.

4.4 Antennen-Tracker-Ausgabe

Telemetrie & Hardware -> Antennen-Tracker / Telemetrie-Ausgabe... sendet die Live-Position laufend an ein externes Antennen-Tracker-Gerät weiter - nützlich, um eine gerichtete Empfangsantenne automatisch dem Modell nachführen zu lassen. Zwei Formate stehen zur Wahl:

- **MAVLink** (GLOBAL_POSITION_INT) - das Format, das z. B. ArduPilot-Antenna-Tracker-Firmware erwartet.
- **NMEA** (\$GPGGA) - ein weit verbreitetes GPS-Sentenzformat mit Prüfsumme, das viele einfachere Tracker verstehen.

Beide Formate lassen sich wahlweise über einen seriellen Port oder per UDP senden. Start und Stopp erfolgen direkt im Dialog, ohne die App neu zu starten; die Ausgabe läuft dann bei jedem eingehenden Telemetrie-Update automatisch mit.

4.5 Modell-Profil

Wer mehrere Modelle mit unterschiedlichen Akkus und Dashboard-Vorlieben fliegt, kann diese unter Telemetrie & Hardware -> Modell-Profil verwalten... als benannte Profile speichern: jedes Profil bündelt Akku-Chemie, Zellenzahl, die genauen Warn-/Kritisch-Spannungen, die Nennkapazität des Akkus (mAh), Fahrzeugtyp (Quadrocopter/Wing/Flugzeug) sowie die aktuelle Dashboard-Feldauswahl und -Anordnung. Ein Klick auf Laden wendet ein gespeichertes Profil sofort an - genau dieselben Einstellungen, die auch die Dialoge Akkuwarnung... und Dashboard anpassen... einzeln setzen würden.

Schneller geht es über das Modellauswahl-Dropdown direkt oben in der Telemetrie-Leiste (siehe Abschnitt 6.2): ein gespeichertes Profil auswählen wendet sofort dessen Akku-Schwellwerte auf Anzeige und Sprachwarnung

an, ohne den Dialog überhaupt zu öffnen. Der kleine Stift-Button daneben öffnet den erweiterten Modell-Editor direkt für das gewählte Profil (Akku-Chemie, Zellenzahl, Fahrzeugtyp und Geofence-Werte auf einen Blick). Der Eintrag "+ Neues Modell anlegen" im selben Dropdown öffnet direkt den Modell-Editor zum Anlegen eines neuen Profils.

5. Offline-Nutzung (Longrange ohne Internet)

Die App ist für den Feldeinsatz gebaut, wo oft kein Internet zur Verfügung steht. Telemetrieempfang, Dashboard, künstlicher Horizont, Wegpunkt-Planung/-Editor, Sperrzonen-Anzeige und -Distanzwarnung, Sprachwarnungen, Fluglog, Track-Aufzeichnung, Antennen-Tracker-Ausgabe und Modell-Profile funktionieren vollständig ohne Internetverbindung - auch die Karte selbst ist fest in die App eingebettet und lädt nicht mehr von einem CDN nach. Drei Dinge brauchen ursprünglich eine Verbindung, werden inzwischen aber alle auf die Festplatte gecacht und danach auch offline aus dem Cache bedient - jeder erfolgreiche Online-Abruf aktualisiert den jeweiligen Cache automatisch für das nächste Mal:

- **Kartenkacheln** (OpenStreetMap/Satellit) - jede einmal angezeigte Kachel landet unter `~/.elrs_ground_station/tile_cache` und wird beim nächsten Aufruf, auch offline, von dort geladen statt erneut vom Kartenserver abgerufen zu werden. Nur Gebiete, die noch nie online angezeigt wurden, bleiben ohne Internet leer/grau.
- **Höhenprofil der Route** (Open-Elevation-Abfrage, siehe Abschnitt 8.4) - bereits abgefragte Punkte werden unter `~/.elrs_ground_station/elevation_cache.json` gecacht; nur wirklich neue Punkte brauchen einen erneuten Online-Abruf, schlägt dieser fehl, erscheint im Dialog eine Fehlermeldung statt eines Absturzes.
- **OpenAIP-Sperrzonen laden** (siehe Abschnitt 7.2) - die zuletzt heruntergeladenen Zonen für eine Region werden unter `~/.elrs_ground_station/openaip_cache.json` gecacht; schlägt ein erneuter Download fehl, werden automatisch die zwischengespeicherten Zonen weiterverwendet.

Empfohlen für den Longrange-Einsatz: die App vor der Abfahrt einmal zuhause mit Internetverbindung im geplanten Fluggebiet öffnen (Karte ansehen, Höhenprofil/OpenAIP-Zonen laden), damit die Caches gefüllt sind und im Feld alles ohne Verbindung verfügbar ist.

Die Vektorkarte (siehe Abschnitt 7.1, der Standard-Kartentyp) hat eine andere Offline-Logik als die Raster-Karte oben: eine Regions-Datei enthält von vornherein die kompletten Kartendaten für das ganze Land, es muss also - anders als beim Kachel-Cache oben - vorher gar nichts online besucht werden, sobald die Region einmal heruntergeladen ist. Der Download selbst (Anzeige & Karte -> Kartentyp -> Vektorkarten-Region herunterladen...) braucht naturgemäß einmalig eine Internetverbindung; danach ist die Region dauerhaft offline nutzbar, ohne erneuten Download.

6. Die Benutzeroberfläche

6.1 Die Karte

Zeigt die Live-Position des Modells auf der Karte (Vektorkarte als Standard, alternativ OpenStreetMap oder Esri-Satellitenbild - Anzeige & Karte -> Kartentyp), mit:

- Nachgezogenem, orangefarbenem Flugpfad seit Verbindungsbeginn.
- Einem Häuschen-Symbol an der Home-Position (dem ersten empfangenen GPS-Fix der laufenden Sitzung).
- Einem wählbaren Fahrzeugsymbol (Quadrocopter, Wing, Flugzeug), das sich mit der Kompassrichtung mitdreht.
- **Auto-Center**: die Karte folgt automatisch der aktuellen Position - abschaltbar per Menü oder per Klick auf den Lock-Button direkt auf der Karte (Google-Maps-Stil).
- **Kartenausrichtung**: ein zweiter fixer Kartenbutton schaltet zwischen Norden-oben und Drohnenrichtung-oben um. Im letzteren Modus dreht sich die gesamte Karte kontinuierlich mit dem aktuellen Kurs, während das Drohnensymbol selbst immer nach oben zeigt.

- Einer optionalen, per Klick oder Rechtsklick gezeichneten bzw. importierten Route (grün, gestrichelt, nummerierte Wegpunkte).
- Optional angezeigten No-Fly-Zones (rote Kreise/Polygone, siehe Abschnitt 7.2).

Home-Symbol, Wegpunkt-Nummern, Segment-Distanzangaben und Sperrzonen-Namen bleiben bei gedrehter Karte stets aufrecht und lesbar. Bei den Raster-Karten (OpenStreetMap/Satellit) drehen sich die in die Kartenkacheln selbst eingebrannten Orts-/Straßennamen zwangsläufig mit der Karte mit, da sie Teil des Kachelbilds sind - das betrifft jede rasterbild-basierte Kartendrehung, auch bei Luftfahrt- und Marine-Navigationsgeräten. Bei der Vektorkarte (siehe Abschnitt 7.1, Standard) bleiben auch die Orts-/Straßennamen selbst aufrecht.

6.2 Das Dashboard

Oben in der Telemetrie-Leiste sitzt ein Modellauswahl-Dropdown: es listet alle gespeicherten Modell-Profile (siehe Abschnitt 4.5), eine Auswahl wendet sofort die passenden Akku-Schwellwerte an, und ein Eintrag "+ Neues Modell anlegen" öffnet direkt den Modell-Editor. Darunter zeigt die Telemetrie-Leiste alle Telemetriedaten gruppiert an:

Gruppe	Felder
GPS	Breitengrad, Längengrad, Höhe, Satellitenanzahl
Status	Flugmodus
Link	RSSI, Link-Qualität (LQ), SNR, Sendeleistung
Akku	Spannung, Restkapazität, Min-Zellspannung, Strom, verbrauchte Kapazität (mAh)
Sensoren	Vario, Baro-Höhe, RPM, Temperatur
Long-Range	Geschwindigkeit, Wind, Entfernung/Peilung zur Home-Position, Flugzeit, Energiebudget, Azimut/Elevation (Boden)
Verbindung	Status-LED + Text (Verbunden/Getrennt)

Jedes einzelne Feld - nicht nur ganze Gruppen - lässt sich über Einstellungen -> Dashboard anpassen... (auch unter Anzeige & Karte gespiegelt) ein- oder ausblenden. Der gleiche Dialog erlaubt zusätzlich, die Reihenfolge der Gruppen per Ziehen neu zu sortieren, festzulegen, auf wie viele Zeilen bzw. Spalten (1-4) sie verteilt werden - praktisch, wenn viele Felder gleichzeitig sichtbar sein sollen - und zu wählen, an welcher Seite des Fensters das Dashboard andockt ist: oben, unten, links oder rechts (Standard). Die Startgröße beträgt dabei ca. 20% der Fensterbreite (bzw. -höhe bei oben/unten-Andockung), wächst aber automatisch mit, falls die gewählte Spaltenzahl mehr Platz braucht; wie bei jedem anderen Fenster-Trennbalken lässt sich die Größe des Dashboard-Bereichs danach jederzeit frei mit der Maus ziehen. Passt der gesamte Inhalt nicht in die verfügbare Fensterhöhe, wird das Dashboard vertikal scrollbar, statt das Fenster über den Bildschirm hinaus zu vergrößern. Sichtbarkeit, Reihenfolge, Zeilenanzahl und Andock-Position werden als persönlicher Standard in `~/elrs_ground_station/dashboard_fields.json`, `dashboard_layout.json` bzw. `dashboard_position.json` gespeichert und bei jedem weiteren Start automatisch wieder geladen. Ist das Dashboard links oder rechts andockt, ordnen sich die Felder innerhalb jeder Gruppe automatisch untereinander statt nebeneinander an, damit sie in die schmalere Spalte passen.

Zusätzlich lässt sich unter Telemetrie & Hardware -> Dashboard-Größe die Schrift-, Icon- und Abstandsgröße der gesamten Telemetrie-Leiste in drei Stufen (Klein 75%, Mittel 100%, Groß 125%) skalieren. Beim allerersten Start wird automatisch ein sinnvoller Wert anhand der tatsächlichen Bildschirmgröße vorbelegt (kompaktere Voreinstellung auf 1920x1080-Displays, großzügigere auf 2K/4K-Bildschirmen) - die eigene Auswahl über das Menü hat danach immer Vorrang.

Künstlicher Horizont und Höhenverlauf sind standardmäßig direkt oben in die Telemetrie-Leiste eingebettet (nebeneinander in einer eigenen Zeile über den Feldgruppen), lassen sich aber jederzeit wieder als freie Karten-Overlays lösen; der Wegpunkt-Editor lässt sich wahlweise unterhalb der Feldgruppen andocken - siehe Abschnitt 6.3.

6.3 Verschieb- und größenveränderbare Karten-Overlays

Der künstliche Horizont, der Wegpunkt-Editor, die Tracking-Aufzeichnung und der Höhenverlauf (siehe 6.6) erscheinen als eigene Panels direkt auf der Karte - wie kleine Fenster, nicht als separate Dialoge. Jedes davon:

- lässt sich frei verschieben - einfach an einer nicht-interaktiven Stelle (Titelzeile) anklicken und ziehen,
- lässt sich an einer kleinen Anfassmarke in der unteren rechten Ecke mit der Maus größer oder kleiner ziehen, wie bei einem Fenster,
- lässt sich über ein kleines Schließen-Symbol (x) in der oberen rechten Ecke ausblenden - der zugehörige Menüpunkt wird dabei automatisch abgehakt entfernt, sodass es von dort wieder eingeblendet werden kann,
- lässt sich über Anzeige & Karte bzw. das jeweilige Menü komplett ein-/ausblenden.

6.4 Künstlicher Horizont

Zeigt Roll und Pitch als klassischer Fluglageanzeiger, gespeist aus MAVLink- oder CRSF-Attitude-Daten. Standardmäßig im Telemetrie-Panel angedockt (Größe passt sich dabei automatisch der Panel-Breite an); lässt er sich lösen, bietet Anzeige & Karte zusätzlich feste Ecken-Presets (oben links/rechts, unten links/rechts) sowie feste Größenstufen (75%-200%) für die freie Positionierung auf der Karte.

6.5 RSSI/LQ-Heatmap

Anzeige & Karte -> RSSI/LQ Heatmap aktivieren färbt den live geflogenen Pfad nach der aktuellen Verbindungsqualität ein - grün (LQ $\geq$ 80%), gelb (LQ $\geq$ 50%) oder rot (darunter), grau, solange kein Wert vorliegt.

6.6 Live-Höhenverlauf

Anzeige & Karte -> Höhenverlauf anzeigen blendet ein Diagramm ein (standardmäßig im Telemetrie-Panel eingebettet), das die tatsächlich geflogene Höhe fortlaufend über die verstrichene Zeit aufzeichnet. Die Zeiteinheit der X-Achse lässt sich zwischen Sekunden, Minuten und Stunden umschalten.

7. Kartenoptionen

7.1 Kartentyp: Vektorkarte (Standard), OpenStreetMap oder Satellit

Anzeige & Karte -> Kartentyp schaltet zwischen der Vektorkarte (MapLibre, Standard) und den klassischen Raster-Layern OpenStreetMap sowie Esri-Satellitenbild um.

Die Vektorkarte nutzt - im Gegensatz zu OpenStreetMap/Satellit - keine Bildkacheln, sondern echte Vektor-Kartendaten (MapLibre GL). Die Kartendrehung bei Drohnenrichtung-oben läuft dabei nativ, und selbst die Orts-/Straßennamen auf der Karte selbst bleiben aufrecht/lesbar - die einzige Einschränkung der Raster-Karte (siehe Hinweis in Abschnitt 6.1), die es hier nicht gibt. Live-Position, Wegpunkt-Bearbeitung, Sperrzonen und RSSI/LQ-Heatmap funktionieren genauso wie bei der Raster-Karte; nur der Kartentyp-Wechsel selbst braucht einen Neustart der App.

Kartendaten für die Vektorkarte kommen aus lokalen Regions-Dateien (automatisch anhand der Home-Position ausgewählt, siehe auch Abschnitt 5 zur Offline-Nutzung). Diese Dateien lassen sich direkt in der App herunterladen: Anzeige & Karte -> Kartentyp -> Vektorkarten-Region herunterladen... öffnet einen Dialog mit einer Länderliste (Ankreuzfeld pro Land, Suchfeld, Alle auswählen/Auswahl aufheben) - aktuell knapp 38 europäische Länder/Regionen, von Portugal bis in die Ukraine, von Island bis Griechenland. Mehrere Länder lassen sich gleichzeitig ankreuzen und werden dann automatisch nacheinander heruntergeladen; pro Region lädt die App per HTTP-Range-Requests nur deren eigene Kacheln aus Protomaps' öffentlichem Kartendaten-Archiv - nicht die komplette Weltkarte. Ein Fortschrittsbalken zeigt den Downloadstand der aktuellen Region (bei mehreren Regionen zusätzlich "Region X/N"), ein Klick auf Abbrechen bricht die laufende Region ab und überspringt alle noch nicht gestarteten. Der Dialog ist nicht-modal: die restliche App (Karte, Telemetrie, Fliegen) bleibt während eines laufenden Downloads voll bedienbar - der eigentliche Download läuft ohnehin schon in einem eigenen Hintergrund-Thread, nicht auf dem GUI-Thread. Ist noch keine passende Region vorhanden, bleibt die Karte beim Start leer und ein Dialog erklärt, wie eine heruntergeladen wird. Es gibt

kein automatisches Hintergrund-Update - bei Bedarf einfach erneut über denselben Menüpunkt herunterladen.

Sowohl beim Start aus dem Quellcode (python main.py) als auch in der kompilierten .exe funktioniert die Vektorkarte identisch, sobald eine Region heruntergeladen wurde (siehe auch Abschnitt 14). Die neu hinzugekommenen europäischen Regionen jenseits von Deutschland/Österreich/Schweiz/Italien nutzen näherungsweise Bounding-Boxen (nicht einzeln gegen echte Regions-Header verifiziert) - der reale Download-Ausschnitt kann dadurch am Rand geringfügig größer oder kleiner ausfallen als das jeweilige Land.

7.2 No-Fly-Zones, Distanz-Warnung und OpenAIP

Alle Sperrzonen-Funktionen sitzen im Untermenü Anzeige & Karte -> Sperrzonen (nicht als eigene, oberste Menügruppe), da sie inhaltlich zur Kartendarstellung gehören.

Anzeige & Karte -> Sperrzonen -> Sperrzonen laden... importiert Sperrzonen aus einer GeoJSON- oder CSV-Datei und zeigt sie als rote Kreise (CSV mit Radius) bzw. Polygone (GeoJSON Polygon/MultiPolygon) auf der Karte an. Sperrzonen anzeigen blendet sie ein/aus, ohne die geladenen Zonen zu verwerfen.

Distanz-Warnung aktivieren (50m) löst - sobald das Modell sich einer geladenen Sperrzone auf 50 Meter nähert - eine Sprachwarnung sowie eine Meldung in der Statusleiste aus. Zusätzlich zeigt ein zentraler Warnbanner auf der Karte alle aktuell aktiven Warnungen (Akku, Sperrzonen, Energiebudget) gebündelt an.

OpenAIP-Einstellungen... hinterlegt einen optionalen OpenAIP-API-Key sowie die gewünschten Luftraumtypen (z. B. CTR, Restricted, Prohibited). OpenAIP Zonen laden lädt anschließend automatisch passende Luftraumdaten für die aktuelle Home-Position herunter und zeigt sie wie manuell importierte Sperrzonen an.

7.3 Rechtsklick-Menü

Ein Rechtsklick auf die Karte öffnet - unabhängig vom Wegpunkt-Modus - ein Kontextmenü mit folgenden Punkten:

- **Wegpunkt / Startpunkt / Endpunkt** - fügt an der angeklickten Stelle einen entsprechenden Punkt zur Route hinzu (siehe Abschnitt 8).
- **Als Home setzen** - teucht die Home-/Startposition (siehe 7.5) direkt an der angeklickten Stelle, ohne einen Dialog öffnen zu müssen.
- **Ansicht** (Untermenü) - Schnellzugriff auf Auto-Center, Kartenausrichtung, Wegpunkt-Editor anzeigen, Koordinaten anzeigen und RSSI/LQ-Heatmap.

7.4 Koordinatenanzeige

Anzeige & Karte -> Koordinaten anzeigen blendet ein kleines Overlay ein, das Lat/Lon direkt neben dem Mauszeiger anzeigt, während dieser sich über der Karte bewegt. Standardmäßig ausgeblendet.

7.5 Home-/Startposition und Bodenstations-Position

Einstellungen -> Home-Position... legt fest, wo die Karte beim nächsten Start zentriert ist - unabhängig von der live ermittelten Flugstart-Referenz (immer der erste GPS-Fix der laufenden Sitzung), die für die Entfernungs-/Peilungsanzeige im Dashboard verwendet wird. Eine dritte, ebenfalls unabhängige Position ist die eigene Bodenstations-Position (Einstellungen -> Bodenstations-Position..., manuelle Lat/Lon/Höhe-Eingabe) - sie dient als Referenzpunkt für die Azimut/Elevation-Anzeige im Dashboard, gedacht zum manuellen Ausrichten einer Richtantenne.

7.6 Karten-Performance

Die Karte ist auf flüssige Darstellung auch bei hoher Telemetrierate optimiert: GPU-beschleunigtes Rendering, ein 500-MB-Festplatten-Cache und eine auf maximal 5 Hz gedrosselte Positionsaktualisierung. Einstellbar über Anzeige & Karte -> Karten-Performance...

8. Route/Wegpunkte planen, importieren und als INAV-Mission exportieren

Neben der live aufgezeichneten Flugspur (orange) kann eine unabhängige, geplante Route (grün, gestrichelt, mit nummerierten Wegpunkt-Markern) auf der Karte angezeigt werden - entweder von Hand gezeichnet (Route & Planung -> Wegpunkt-Modus, oder per Rechtsklick -> Wegpunkt/Startpunkt/Endpunkt) oder importiert aus einer Datei (Route & Planung -> Route importieren...).

8.1 Unterstützte Import-/Exportformate

Format	Beschreibung
.gpx	Liest bzw. schreibt Routenpunkte (<rte>); beim Import ersatzweise Track-Punkte oder einzelne Wegpunkte.
.mission (JSON)	Modernes INAV-Missionsformat (Schema-Version 1.0), siehe Abschnitt 8.3.
.mission (XML)	Älteres 'MW XML'-Missionsformat von mwp/iNav Configurator/ezgui.
.xml	Generischer Fallback: durchsucht beliebige XML-Strukturen nach erkennbaren Lat/Lon/Alt/Name-Bezeichnungen.
.csv	Erkennt Spalten wie lat/latitude, lon/longitude, optional alt und name.

8.2 Der Wegpunkt-Editor

Route & Planung -> Wegpunkt-Editor anzeigen blendet ein Overlay direkt auf der Karte ein. Oben zeigt es Wegpunktzahl und Gesamtdistanz, darunter eine Tabelle mit je einer Zeile pro Wegpunkt. Der Editor ist vollständig interaktiv und mit den Wegpunkt-Markern auf der Karte verzahnt (Auswahl, Bearbeiten, Löschen, Verschieben per Maus, Umsortieren per Drag & Drop).

8.3 INAV-Mission (.mission JSON)

Unterstützte Aktionen:

Aktion	Bedeutung von P1/P2/P3
WAYPOINT	P1 = Wartezeit in Sekunden (0 = Durchflug ohne Halt).
HOLD	P1 = Haltedauer in Sekunden.
RTH	Rückkehr zur Home-Position; Lat/Lon/Alt dürfen 0 sein.
SET_POI	Lat/Lon/Alt definieren das Kameraziel.
JUMP	P1 = Ziel-Wegpunktindex (1-basiert), P2 = Wiederholungsanzahl.
LAND	Automatische Landung an Lat/Lon.

8.4 Höhenprofil der Route

Tools & Simulation -> Höhenprofil der Route anzeigen öffnet ein Diagramm, das für die aktuelle Route die Geländehöhe (braun) und die geplante Flughöhe (türkis) über der zurückgelegten Distanz darstellt.

8.5 Grid-/Suchmuster-Generator

Route & Planung -> Grid-/Suchmuster erzeugen... erstellt automatisch eine Zickzack-Absuchroute (Boustrophedon-Muster) über zwei Eckpunkte oder Mittelpunkt+Radius.

9. Flugpfad-Aufzeichnung (Start/Pause/Export)

Ein eigenes Overlay auf der Karte startet und pausiert die Aufzeichnung des geflogenen Pfads unabhängig von der Live-Anzeige. Exportieren... im Overlay fragt zunächst das gewünschte Format ab - GPX, KML oder CSV.

Diese Aufzeichnung ist unabhängig vom Fluglog (Abschnitt 10): das Tracking speichert nur Positionspunkte für einen späteren Pfad-Export, das Fluglog schreibt alle Telemetriefelder kontinuierlich als Zeitreihe in eine CSV-Datei.

10. Fluglog (CSV-Aufzeichnung)

Das Fluglog zeichnet - unabhängig von der Flugpfad-Aufzeichnung aus Abschnitt 9 - sämtliche Telemetriedaten als Zeitreihe in einer CSV-Datei auf. Über Telemetrie & Hardware -> Log-Einstellungen... lässt sich sowohl die Menge der aufgezeichneten Spalten als auch das Aufzeichnungsintervall (0,1 bis 60 Sekunden) frei wählen. Eine aufgezeichnete Log-Datei lässt sich später über Tools & Simulation -> Flug abspielen... wieder einladen und wie eine Live-Verbindung durch die App abspielen (Log-Replay), inklusive Wiedergabegeschwindigkeit, Sprungmarke und einer abschließenden Flugzusammenfassung.

11. Plan-Modus

Der Plan-Modus erlaubt es, Routen zu planen und Wegpunkte zu setzen, ohne dass irgendeine Telemetrie-Verbindung - echt oder simuliert - läuft. Aktivierbar über Tools & Simulation -> Plan-Modus oder direkt im Verbindungs-Popup beim Programmstart.

12. Die Menüs im Detail

Die Menüleiste ist in sieben Gruppen sortiert: Datei | Route & Planung | Anzeige & Karte | Telemetrie & Hardware | Tools & Simulation | Einstellungen | Hilfe.

12.1 Datei

- Flugpfad als GPX exportieren... / als KML exportieren...
- Beenden

12.2 Route & Planung

- Wegpunkt-Modus, Letzten Wegpunkt entfernen / Route löschen
- Wegpunkt-Editor anzeigen / im Dashboard andocken
- Route importieren... / Route exportieren...
- Grid-/Suchmuster erzeugen...
- Mission hochladen... / Mission herunterladen... (MAVLink, siehe Abschnitt 4)

12.3 Anzeige & Karte

- Kartentyp -> Vektorkarte (MapLibre, Standard) / OpenStreetMap / Satellit (Esri) (Neustart bei Wechsel erforderlich) sowie Vektorkarten-Region herunterladen... - siehe Abschnitt 7.1.
- Sperrzonen (Untermenü): Sperrzonen laden... / Sperrzonen anzeigen, Distanz-Warnung aktivieren (50m), OpenAIP-Einstellungen... / OpenAIP Zonen laden - siehe Abschnitt 7.2.
- Auto-Center, Drohnenrichtung/Norden oben, Aktuelle Position anspringen (Strg+Pos1).
- Wegpunkt-Editor anzeigen, Karten-Performance..., Tracking-Overlay anzeigen, Höhenverlauf anzeigen, Koordinaten anzeigen, RSSI/LQ Heatmap aktivieren.
- Fahrzeugtyp, Künstlicher Horizont anzeigen/Position/Größe.
- Dashboard anpassen... - hier zur schnellen Erreichbarkeit gespiegelt (siehe Abschnitt 6.2).

12.4 Telemetrie & Hardware

- Verbindung... - wechselt zur Laufzeit zwischen WiFi/UDP und USB/Seriell sowie zwischen MAVLink und CRSF.
- Log-Einstellungen... / Logging aktiv - siehe Abschnitt 10.
- Akkuwarnung... - siehe Abschnitt 13.
- Warntöne... - siehe Abschnitt 13.1.
- MAVLink-STATUSTEXT-Konsole anzeigen - Roh-text-Meldungen der Flugsteuerung, farblich nach Schweregrad.
- Dashboard-Größe -> Klein (75%) / Mittel (100%) / Groß (125%) - skaliert Schrift, Icons und Abstände der gesamten Telemetrie-Leiste; wird beim ersten Start automatisch anhand der Bildschirmgröße vorgelegt (siehe Abschnitt 6.2).
- Antennen-Tracker / Telemetrie-Ausgabe... - siehe Abschnitt 4.4.
- Modell-Profil verwalten... - siehe Abschnitt 4.5.
- RTH auslösen / Modus wechseln... (nur bei aktiver MAVLink-Verbindung, mit Bestätigungsdialog).

12.5 Tools & Simulation

- Demo-Modus - schaltet zur Laufzeit zwischen echter Telemetrie und simulierten Daten um.
- Plan-Modus - siehe Abschnitt 11.
- Flug abspielen... - lädt ein aufgezeichnetes Fluglog (Abschnitt 10) und spielt es wie eine Live-Verbindung ab.
- Höhenprofil der Route anzeigen - siehe Abschnitt 8.4.

12.6 Einstellungen

- Home-Position... - siehe Abschnitt 7.5.
- Bodenstations-Position... - siehe Abschnitt 7.5.
- Dashboard anpassen... - siehe Abschnitt 6.2.
- Sprache -> Deutsch/English, wechselt die komplette Oberfläche sofort ohne Neustart.

12.7 Hilfe

- Benutzerhandbuch öffnen... - öffnet dieses Handbuch als PDF im Standard-PDF-Betrachter des Systems.

13. Akkuwarnung: LiPo vs. Li-Ion

Sobald der Akku niedrig oder kritisch niedrig wird, gibt die App eine Sprachwarnung aus (offline, über die Windows-SAPI5-Sprachausgabe). Da sich die sichere Entladeschlussspannung von LiPo- und Li-Ion-Zellen deutlich unterscheidet, lässt sich die Chemie unter Telemetrie & Hardware -> Akkuwarnung... auswählen:

Chemie	Warnung (V/Zelle)	Kritisch (V/Zelle)
LiPo	3,6 V	3,5 V
Li-Ion	3,3 V	3,0 V

Die Auswahl der Chemie füllt automatisch passende Standardwerte vor; Zellenzahl, die genauen Warn-/Kritisch-Spannungen sowie die Nennkapazität des Akkus (mAh) lassen sich zusätzlich frei einstellen. Steht eine echte Zellspannungsmessung zur Verfügung (CRSF-Cells-Frame oder MAVLink BATTERY_STATUS), verwendet die App die tatsächliche niedrigste Zellspannung für die Warnung.

13.1 Warntöne (EdgeTX-Sounds)

Standardmäßig werden alle Warnungen der App - Akku niedrig/kritisch, Geofence verletzt, Sperrzone nähert sich, Umkehrpunkt erreicht, Energiereserve kritisch, Telemetrie verloren - per Sprachausgabe (SAPI5)

angesagt. Über Telemetrie & Hardware -> Warntöne... lässt sich jeder dieser sieben Warnungen stattdessen ein konkreter Sound aus dem mitgelieferten EdgeTX-Sprachpaket zuordnen (assets/en, dieselben System- und Skript-Sounds, mit denen EdgeTX-Sender ausgeliefert werden - ca. 730 Dateien).

Der Dialog zeigt pro Warnung ein durchsuchbares Auswahlfeld (Freitext filtert die Liste) sowie einen Vorhören-Knopf. Änderungen werden sofort übernommen, es gibt keinen OK/Abbrechen-Schritt. Bleibt eine Warnung auf "Sprachausgabe (Standard)" stehen, verhält sie sich unverändert wie bisher. Die Sound-Wiedergabe funktioniert nur unter Windows (über die Windows-eigene winsound-API); unter anderen Betriebssystemen fällt die App automatisch auf die Sprachausgabe zurück.

14. Als eigenständige .exe kompilieren

```
cd elrs_ground_station
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt pyinstaller
pyinstaller --name ELRS_GroundStation --onedir --icon assets/app_icon.ico \
    --add-data "docs;docs" --add-data "assets;assets" main.py
```

--add-data "docs;docs" bündelt das PDF-Handbuch mit in die Exe, damit Hilfe -> Benutzerhandbuch öffnen... es auch dort findet. --add-data "assets;assets" bündelt App-Icon, Logo und das komplette EdgeTX-Soundpaket (assets/en, für Abschnitt 13.1); --icon assets/app_icon.ico setzt zusätzlich das Icon der Exe-Datei selbst.

Das Ergebnis liegt unter dist\ELRS_GroundStation\ELRS_GroundStation.exe. Der gesamte Ordner dist\ELRS_GroundStation (Exe plus _internal-Verzeichnis, ca. 500 MB) muss zusammen weitergegeben werden, nicht nur die .exe-Datei allein.

Die Exe behält bewusst die Konsole (kein --windowed), damit --list-ports, --demo usw. weiterhin normal über die Kommandozeile nutzbar sind; beim Doppelklick öffnet sich zusätzlich ein Konsolenfenster im Hintergrund.

Die Vektorkarte (siehe Abschnitt 7.1) funktioniert auch in der Exe, benötigt dort aber - genau wie beim Start aus dem Quellcode - eine heruntergeladene Regions-Datei (Anzeige & Karte -> Kartentyp -> Vektorkarten-Region herunterladen...); die Regions-Dateien selbst sind mehrere GB pro Land groß und werden bewusst nicht mit --add-data gebündelt. Gesucht wird zuerst unter %USERPROFILE%\elrs_ground_station\pmtiles\ (dort landen auch neu heruntergeladene Regionen), ersatzweise unter dist\ELRS_GroundStation_internal\assets\pmtiles\, falls dort von Hand *.pmtiles-Dateien abgelegt wurden. Fehlt eine passende Datei, erklärt ein Dialog beim Start, wo eine hingehört, und die Karte bleibt bis dahin leer.

15. Anhang: Kommandozeilen-Referenz

Option	Beschreibung
--demo	Im Simulationsmodus starten, keine Hardware nötig.
--protocol {mavlink,crsf}	Telemetrieprotokoll (Standard: mavlink).
--connection {udp,usb}	Transportweg (Standard: udp).
--host	Bind-Adresse für UDP-Empfang (Standard: 0.0.0.0).
--port	UDP-Port (Standard: 14550 MAVLink / 14551 CRSF).
--udp-mode {listen,connect}	listen wartet auf Pakete, connect verbindet aktiv zu Host:Port.
--serial-port	USB/seriell-Port bei --connection usb, z. B. COM5.
--baud	Baudrate bei USB (Standard: 57600 MAVLink / 420000 CRSF).
--list-ports	Verfügbare USB/seriell-Ports auflisten und beenden.
--cells	Anzahl LiPo/Li-Ion-Zellen für die Akku-Warnschwellen.
--low-cell-voltage	Zellspannung für die "niedrig"-Warnung.

--critical-cell-voltage	Zellspannung für die "kritisch"-Warnung.
--demo-center lat,lon	Mittelpunkt der Demo-Flugbahn.
--lang {de,en}	Startsprache der Oberfläche.

Vollständige Projektdokumentation, Quellcode und Architektur-Details:
github.com/KresserSimon/ELRS_Telemetry_Groundcontrol